COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	7/24/26	**DATE SUBMITTED:**	7/27/26
NAME:	Gabriel Dwyane D. Arididdon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **HOA 3.1 Linked Lists**

1. PROCEDURE OUTPUT

Screenshot

```
%%writefile main.cpp
#include<iostream>
#include<utility>

class Node{
public:
  char data;
  Node *next;
};

void ListTraversal(Node* head) {
  while(head != nullptr) {
    std::cout << head->data << " ";
    head = head->next;
  }
}


int main() {
//step 1
  Node *head = NULL;
  Node *second = NULL;
  Node *third = NULL;
  Node *fourth = NULL;
  Node *fifth = NULL;
  Node *last = NULL;
//step 2
  head = new Node;
  second = new Node;
  third = new Node;
  fourth = new Node;
  fifth = new Node;
  last = new Node;
//step 3
  head->data = 'C';
  head->next = second;
  second->data = 'P';
  second->next = third;
  third->data = 'E';
  third->next = fourth;
  fourth->data = '0';
  fourth->next = fifth;
  fifth->data = '1';
  fifth->next = last;
//step 4
  last->data = '0';
  last->next = nullptr;

ListTraversal(head);

return 0;


}
```

	
Discussion	Based on what I've observed, the program create a simple linked list by dynamically allocating each node and it connects into a sequence to form a string "C P E 0 1 0". The ListTraversal function then travels to each node starting from the head then prints the node's data until in reaches nullptr.

Table 3-1. Output of Singly Linked List

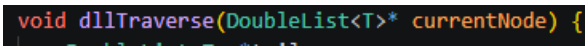
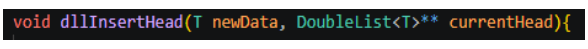
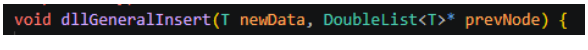
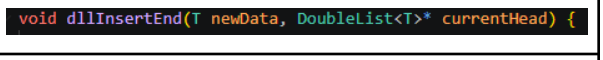
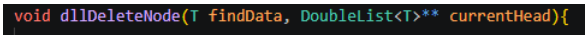
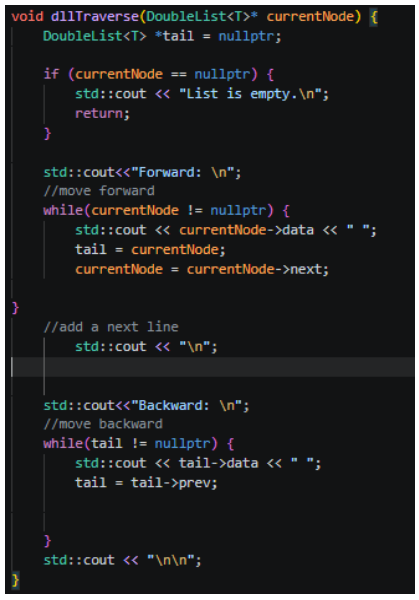
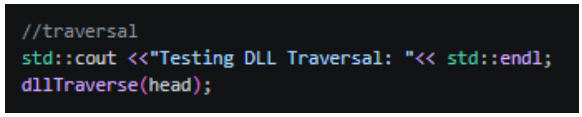
Operation	**Screenshot**
Traversal	
Insertion at head	
Insertion at any part of the list	
Insertion at the end	
Deletion of a node	

Table 3-2. Code for the List Operations

a.	**Source Code**	 

```
void dllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T> *tail = nullptr;

    if (currentNode == nullptr) {
        std::cout << "List is empty.\n";
        return;
    }

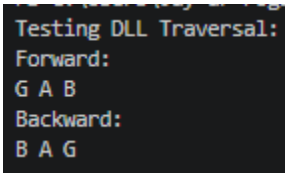
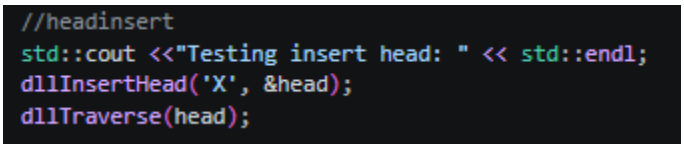
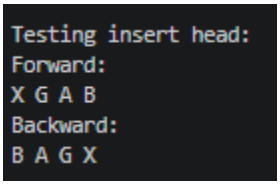
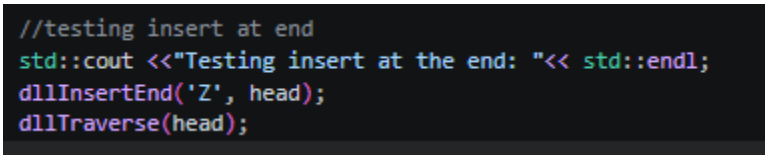
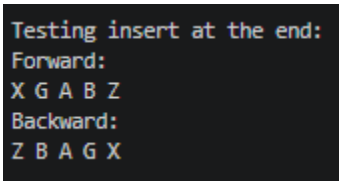
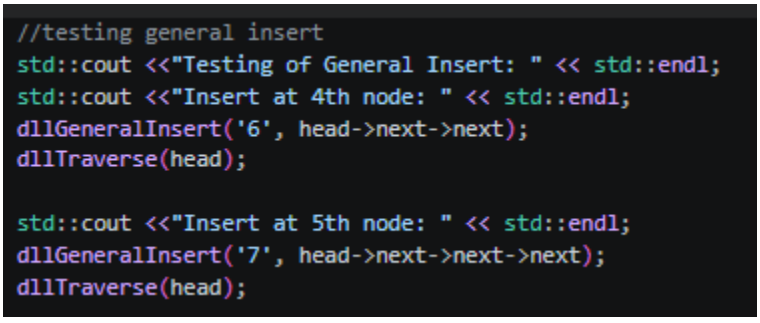
    std::cout<<"Forward: \n";
    //move forward
    while(currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;

}
    //add a next line
        std::cout << "\n";


    std::cout<<"Backward: \n";
    //move backward
    while(tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;


    }
    std::cout << "\n\n";
}
```

```
//traversal
std::cout <<"Testing DLL Traversal: "<< std::endl;
dllTraverse(head);
```

	Console	
b.	**Source Code**	
	Console	
c.	**Source Code**	
	Console	
d.	**Source Code**	

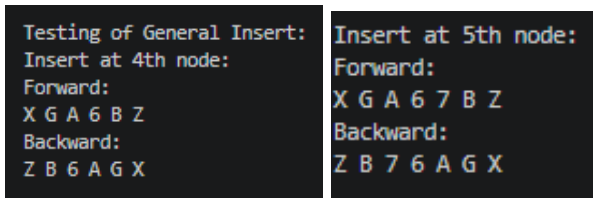
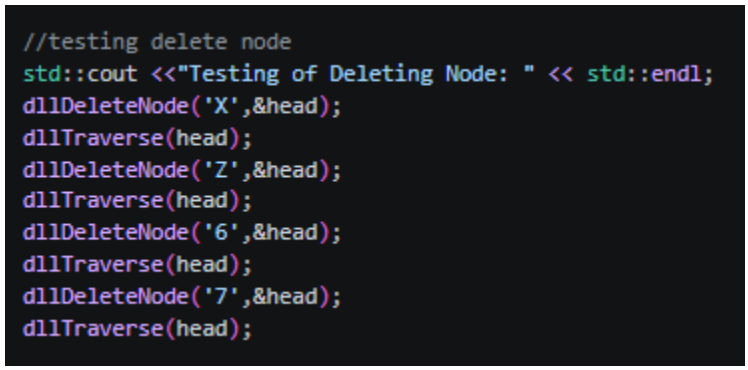
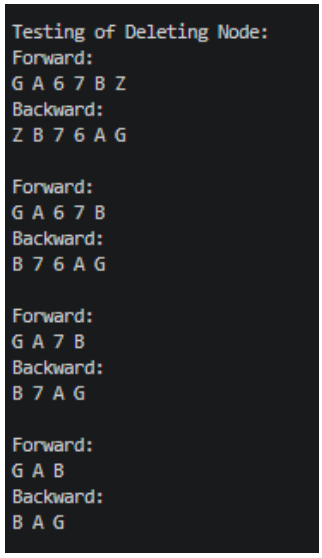
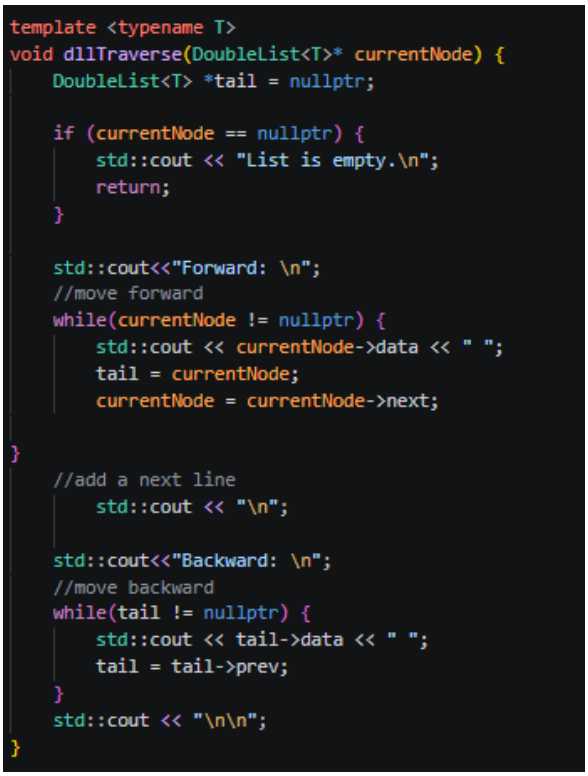
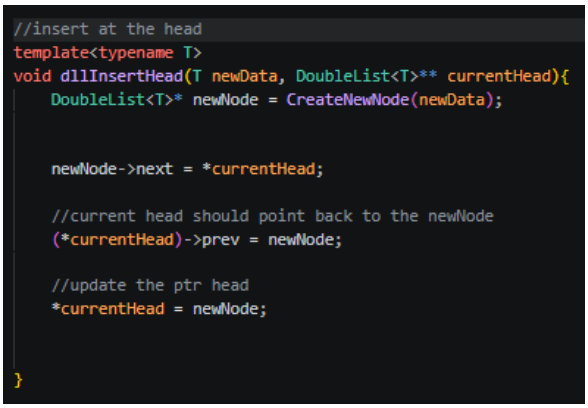
	Console	
e.	**Source Code**	
	Console	

Table 3-3. Code Testing for DoublyLinked Lists

Screenshot	Analysis
	While working on the traversal function, I observed that a doubly linked list is more flexible compared to a singly linked list because it allows me to move through the nodes in both forward and backward directions. In a singly linked list, traversal is only possible using the next pointer, meaning I can only move from the head toward the end of the list. However, in a doubly linked list, the additional prev pointer allows me to access previous nodes without restarting from the head.
	In the insert head operation, I learned that adding a new node at the beginning of a doubly linked list requires updating more pointers compared to a singly linked list. In a singly linked list, I only need to update the new node's next pointer and change the head pointer. However, in a doubly linked list, I also need to update the previous head node's prev pointer so that it points back to the new node.

```
template<typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead) {
    DoubleList<T>* newNode = CreateNewNode(newData);

    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }


    currentHead->next = newNode;


    newNode->prev = currentHead;

}
```

While implementing the insert end operation, I learned how a new node is added after the last node of the list. Similar to a singly linked list, the list must be traversed until reaching the final node, but a doubly linked list requires an additional step of connecting the new node back to the previous node using the prev pointer. In a singly linked list, the new node only needs a next connection because there is no backward link.

```
// general insert
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode) {

    // Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        std::cout << "Previous value cannot be null\n";
        return;
    }
    // Allocate a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    // Store data in the new node
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    // Link new node's next to previous node's next
    newNode->next = prevNode->next;

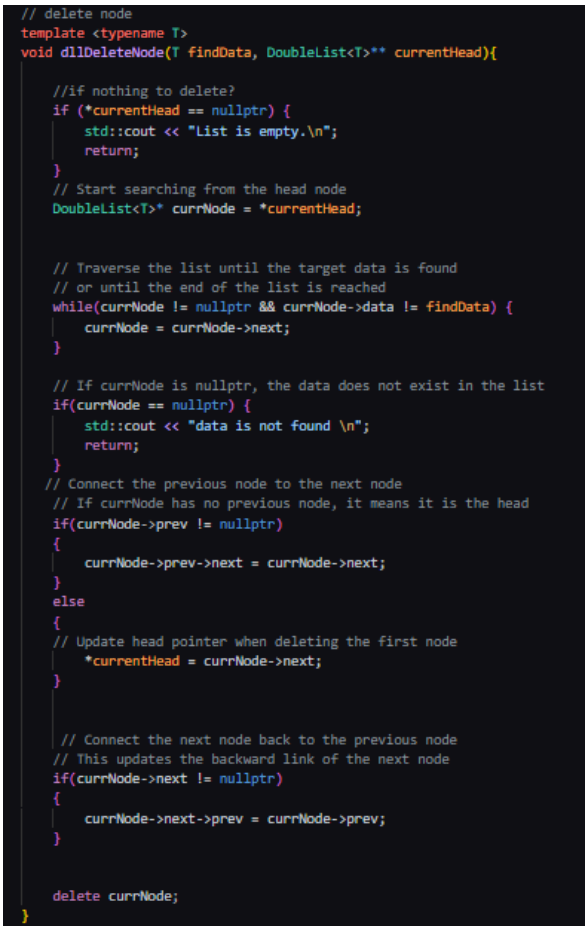
    // Link previous node's next to new node
    prevNode->next = newNode;

    // Link new node's prev to previous node
    newNode->prev = prevNode;

    if (newNode->next != nullptr) {
        newNode->next->prev = newNode;
    }

}
```

The general insert operation helped me understand the difference between inserting nodes in singly and doubly linked lists. In a singly linked list, inserting a node only requires changing the previous node's next pointer and assigning the new node's next pointer. However, in a doubly linked list, it is much more complex since I also need to update the prev pointer of the new node and the next node to maintain both connections.

	While creating the delete node function, I learned that deleting a node in a doubly linked list is different from a singly linked list because each node already stores the address of its previous and next nodes. In a singly linked list, deleting a node requires keeping track of the previous node separately because there is no way to move backward. In a doubly linked list, the prev pointer makes it easier to reconnect the surrounding nodes by directly accessing the previous node. However, I also observed that more pointers must be updated during deletion to avoid breaking the links of the list.

```
// delete node
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** currentHead){

    //if nothing to delete?
    if (*currentHead == nullptr) {
        std::cout << "List is empty.\n";
        return;
    }
    // Start searching from the head node
    DoubleList<T>* currNode = *currentHead;


    // Traverse the list until the target data is found
    // or until the end of the list is reached
    while(currNode != nullptr && currNode->data != findData) {
        currNode = currNode->next;
    }

    // If currNode is nullptr, the data does not exist in the list
    if(currNode == nullptr) {
        std::cout << "data is not found \n";
        return;
    }
   // Connect the previous node to the next node
    // If currNode has no previous node, it means it is the head
    if(currNode->prev != nullptr)
    {
        currNode->prev->next = currNode->next;
    }
    else
    {
    // Update head pointer when deleting the first node
        *currentHead = currNode->next;
    }


     // Connect the next node back to the previous node
    // This updates the backward link of the next node
    if(currNode->next != nullptr)
    {
        currNode->next->prev = currNode->prev;
    }


    delete currNode;
}
```

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List

Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.

- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Operations	Screenshot
Traversal	`void playlistTraverse(Playlist<T>* head){`
Insert End	`void playlistInsertEnd(T newData, Playlist<T>* currentHead){`
Delete Node	`void playlistDeleteNode(T findData, Playlist<T>** currentHead){`
Next Song	`Playlist<T>* playlistNext(Playlist<T>* current){`
Previous Song	`Playlist<T>* playlistPrev(Playlist<T>* current){`

Table 3-5. Code for the Playlist Operations

a.	**Source**	`//traversal`
		`std::cout <<"Testing Playlist Traversal: "<< std::endl;`
		`playlistTraverse(head);`

	Console	`Testing Playlist Traversal:`
		`=====PLAYLIST=====`
		`[1] ITZY - Mr.Vampire`
		`[2] ITZY - DALLA DALLA`
		`[3] ITZY - LOCO`
		`[4] ITZY - In The Morning`
		`[5] ITZY - Icy`
		
		`Playlist finished. Looping back to the first song...`
		
		`[1] ITZY - Mr.Vampire`

b. **Source**

```
//insert end
std::cout << "Testing Insert song:" << std::endl;
playlistInsertEnd(std::string("Silent Sanctuary - Rebound"), head);
playlistTraverse(head);
playlistInsertEnd(std::string("Soapdish - Tensionado"), head);
playlistTraverse(head);
```

Console

```
Testing Insert song:
=====PLAYLIST=====
[1] ITZY - Mr.Vampire
[2] ITZY - DALLA DALLA
[3] ITZY - LOCO
[4] ITZY - In The Morning
[5] ITZY - Icy
[6] Silent Sanctuary - Rebound

Playlist finished. Looping back to the first song...

[1] ITZY - Mr.Vampire
=====PLAYLIST=====
[1] ITZY - Mr.Vampire
[2] ITZY - DALLA DALLA
[3] ITZY - LOCO
[4] ITZY - In The Morning
[5] ITZY - Icy
[6] Silent Sanctuary - Rebound
[7] Soapdish - Tensionado

Playlist finished. Looping back to the first song...
```

c. **Source**

```
//delete node
std::cout << "Testing Delete song:" << std::endl;
playlistDeleteNode(std::string("ITZY - Mr.Vampire"), &head);
playlistTraverse(head);
```

Console

```
Testing Delete song:
=====PLAYLIST=====
[1] ITZY - DALLA DALLA
[2] ITZY - LOCO
[3] ITZY - In The Morning
[4] ITZY - Icy
[5] Silent Sanctuary - Rebound
[6] Soapdish - Tensionado

Playlist finished. Looping back to the first song...

[1] ITZY - DALLA DALLA
```

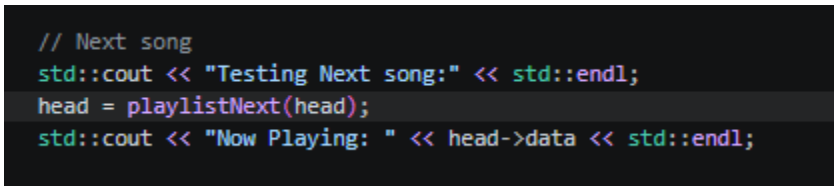
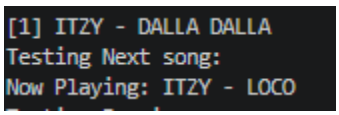
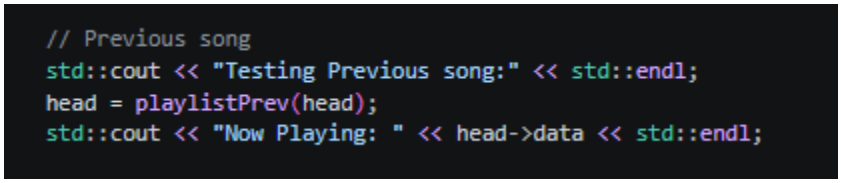
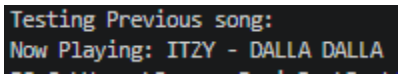
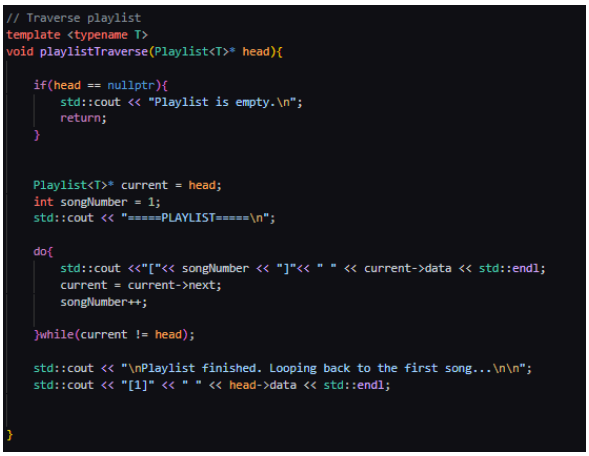
d.	**Source**	
	Console	
e.	**Source**	
	Console	

Table 3-6. Code Testing for Playlist

Screenshots	Analysis
	For the CreateNewSong() function, I applied the same concept as creating a node in a doubly linked list. It dynamically allocates memory for a new node and initializes the data, previous pointer, and next pointer.



```
// Traverse playlist
template <typename T>
void playlistTraverse(Playlist<T>* head){

    if(head == nullptr){
        std::cout << "Playlist is empty.\n";
        return;
    }


    Playlist<T>* current = head;
    int songNumber = 1;
    std::cout << "=====PLAYLIST=====\n";

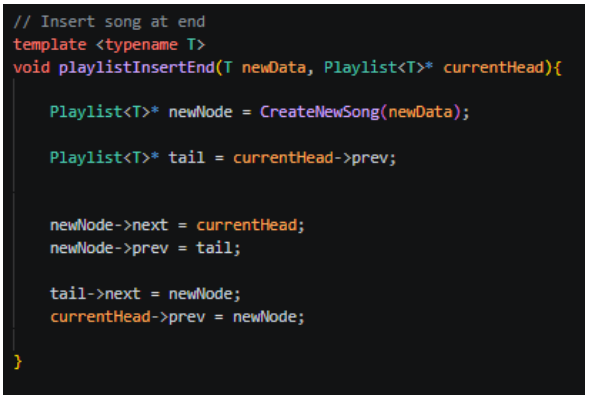
    do{
        std::cout <<"["<< songNumber << "]"<< " " << current->data << std::endl;
        current = current->next;
        songNumber++;

    }while(current != head);

    std::cout << "\nPlaylist finished. Looping back to the first song...\n\n";
    std::cout << "[1]" << " " << head->data << std::endl;


}
```

For the playlistTraverse() function, I observed that the traversal process was different compared to a regular doubly linked list. In my previous doubly linked list, I used a while loop that stops when the pointer reaches nullptr. However, since a circular linked list does not have an ending nullptr, I used a do-while loop that stops once the pointer returns to the starting node. This prevents an infinite loop while allowing all songs to be displayed once.



```
// Insert song at end
template <typename T>
void playlistInsertEnd(T newData, Playlist<T>* currentHead){

    Playlist<T>* newNode = CreateNewSong(newData);

    Playlist<T>* tail = currentHead->prev;


    newNode->next = currentHead;
    newNode->prev = tail;

    tail->next = newNode;
    currentHead->prev = newNode;

}
```

For the playlistInsertEnd() function, I noticed that it was simpler compared to my previous doubly linked list insertion because a circular doubly linked list already stores the tail node using head->prev. Instead of traversing the entire list to find the last node, I can directly access the tail and connect the new song between the tail and the head. This showed me the advantage of circular linked lists because accessing the last element becomes more efficient.

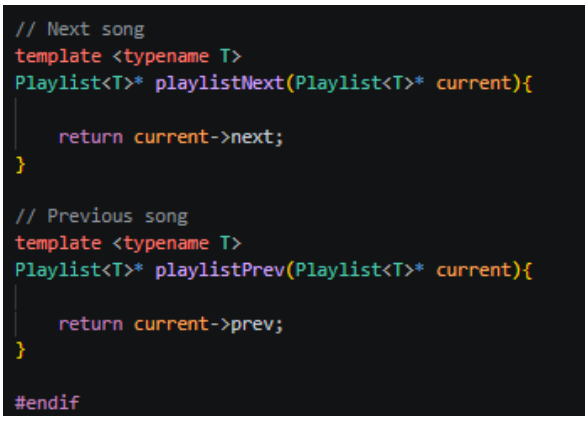
	For the playlistDeleteNode() function, I used my previous doubly linked list delete function as a guide since the process is almost similar. I still needed to find the node that contains the song to be removed and then reconnect the previous and next nodes. The main difference I noticed is that because this is a circular linked list, I had to make sure that the links are still connected after deleting a song. I also had to consider cases like deleting the only song in the playlist or deleting the first song because it affects the head pointer. While making this function, I had some difficulty understanding which pointers needed to be updated, but comparing it with my doubly linked list helped me understand the process better.
	For the playlistNext() and playlistPrev() functions, I observed how the doubly linked list structure becomes useful for applications like music players. Since every node has both next and prev pointers, moving forward and backward between songs is easier compared to a singly linked list where only forward movement is possible.

Table 3-7. Modified Operations for Playlist

Conclusion

This activity helped me understand how linked lists can be applied to real-world problems like creating a music playlist using a circular doubly linked list. I used my previous doubly linked list implementation as a reference when creating the functions, but I had to modify some parts because a circular list requires the last node to connect back to the first node. The most challenging parts for me were the delete node and general insert functions because understanding and updating the pointers was difficult. I received some help from AI, especially for these parts, to better understand the logic and fix errors in my code. Overall, I think I did well in this activity because I was able to apply linked list concepts and create a working playlist system. However, I still need to improve my confidence in

handling pointers and complex node operations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, concept explanation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT to assist me with grammar correction, concept explanation, and code assistance throughout the development of this work. I used the tool especially when I encountered difficulties in implementing delete nodes and general insertion functions. Understanding and visualizing how pointers work was challenging for me, but ChatGPT helped me gain a clearer understanding of the concepts by providing explanations and guidance. I still reviewed and applied the information myself to ensure that I understood the process and that the final work reflected my own learning.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.